MIS686: Term Project

Aviation MRO Enterprise Systems: A Relational Model for Maintenance Operations and Fleet Analytics

Micha Goll

San Diego State University

Fowler College of Business

Department of Management Information Systems

1. Topic, Domain and Industry Relevance

1.1 Introduction to Topic

This term project focuses on creating a so called Maintenance Repair, and Overhaul Enterprise System (MRO). Safety and, being compliant with regulators and operational efficiency are crucial in aerospace. For a long time, MRO providers have relied on scattered single enterprise systems or even logged maintenance manually. This is very risky by today's standards. Delayed repairs means the aircraft is on the ground and can't earn money. In aviation, this matters a lot due to the cost of a plane. Also, proper maintenance is one of the core contributors to flight safety since airplanes are very complex and delicate.

The project aims to tackle these issues through a relational model database centralized on Amazon Web Services. It tracks aircraft by their tail numbers, identifies aircraft models, and captures the maintenance lifecycle. The database enables cost awareness like tracking real cost of ownership of an aircraft and especially fosters aviation security through logging every single bolt and procedure. Nowadays, staying compliant with FAA or EASA regulations is nearly impossible and at the very least not cost-effective.

.

1.2 Business Rules

Fleet Ownership

A single aircraft operator can have several planes under their name, yet every plane links to just one operator only. Known as a one-to-many connection, this 1:N pattern follows a strict rule that matters deeply within the system.

Model Standardization

Aircraft tie to just one model at a time. This keeps specs uniform across the board. On the flip side, one model type might cover several planes within the fleet structure.

Maintenance Documentation

A single aircraft might carry no work orders at all - or several across its operational years. Yet each work order only comes into existence once tied to one particular aircraft.

Task Assignment

A single WorkOrder holds several MaintenanceTasks. The WorkOrderTask table records precise actual_hours for each linked task.

Logistics and Parts Consumption

A single WorkOrder might include several different Parts. Because prices shift over time, trac king each part's cost at usage matters.
The link between orders and parts is stored in the WorkOrderPart table. This record keeps exact figures: how many were used, what they cost then.
Accurate auditing relies on these fixed historical values. Without them, later pricing changes c ould distort past reports.

Operational Status

Aircraft can remain in the hangar only when their associated work order holds an active status. This state - recorded directly in the WorkOrder table - is marked as either Open, In Progress, or Closed. Each value acts as a signal, influencing whether maintenance access continues.

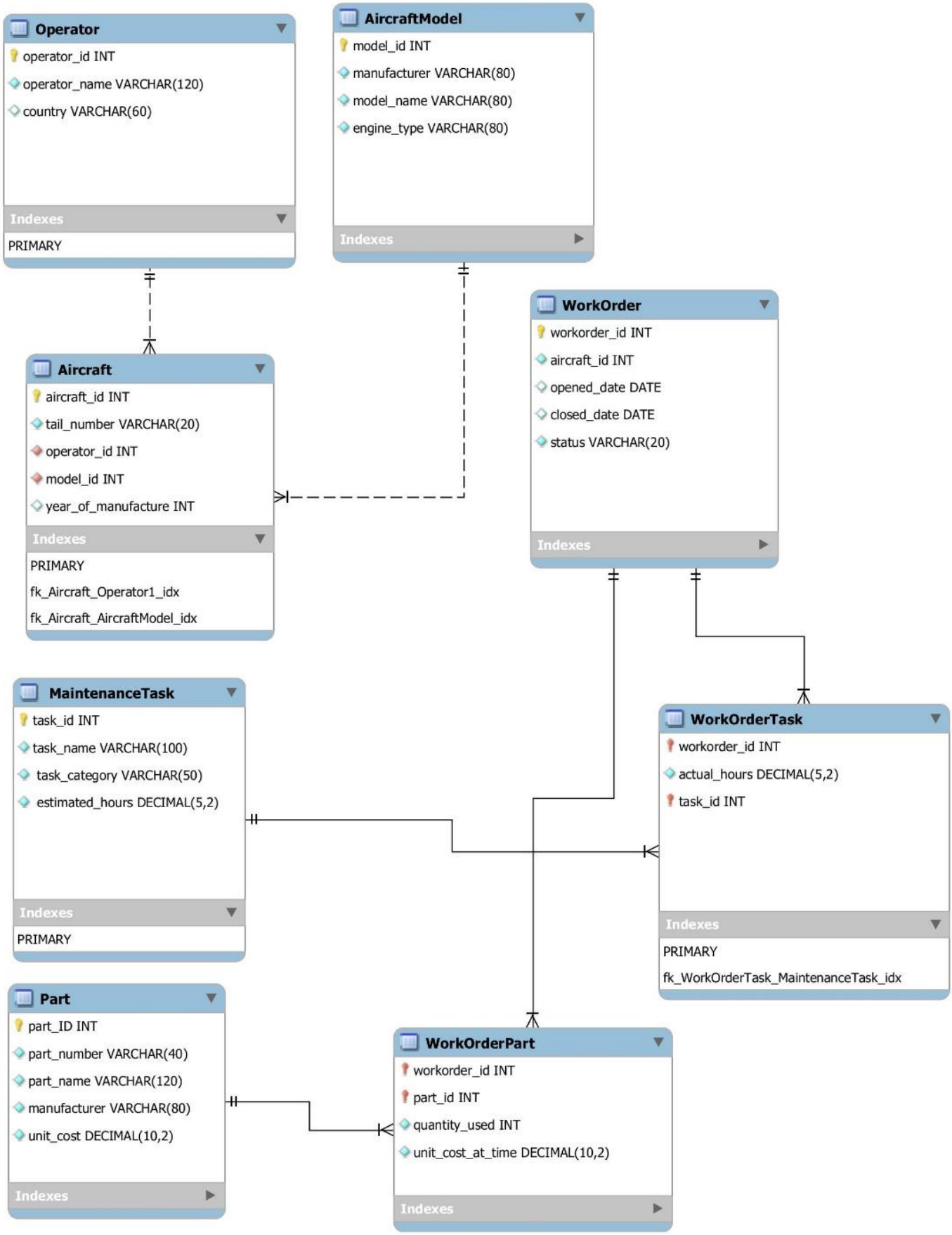
1.3 Use Cases and Industry Areas

Airline companies rely heavily on Aviation MRO systems when overseeing large, expensive fleets - downtime here means steep monetary consequences. Fleet performance monitoring depends on these tools, especially to verify ongoing compliance with rigorous safety standards. Linking operator data with aircraft records makes it possible to plan regular inspections - known as A through D checks - using flight time and scheduled timelines. Planning unfolds ahead of time: maintenance teams align with operations staff to pause service during least disruptive windows. Unexpected standstills called AOG situations drop sharply as a result. More planes stay ready for active routes, supporting consistent commercial operation.
Using information from Part and WorkOrderPart tables, purchasing groups allow visibility into how often certain pieces fail depending on plane type or maker. When patterns in breakdowns emerge, companies find ways to hold less stock without risking downtime - freeing money once locked in unused reserves. Knowing which items wear out fast helps distribution planners adjust what sits in storage centers spread across regions. Availability improves when key elements such as pressure regulators or wheel position detectors are placed near bases with higher demand, guided by past repair activity.
This database supports Labor Analytics and Financial Reporting through detailed tracking. When managers look at the WorkOrderTask table, they see how planned work time compares to what technicians actually spend. Because insights come from real usage patterns, teams

spot delays in operations, adjust invoicing for outside customers, and evaluate how well expert crews perform. Another benefit appears when part costs are logged at purchase price. This gives airline leaders accurate total costs for each plane type's lifetime expense. This enables better decision making.

2. Conceptual Data Modeling and Database Design

2.1 Detailed ERD



Centered on the WorkOrder, the ERD for the Aviation MRE system manages intricate layers of maintenance data across aerospace operations. This core entity ties together physical equipment and the service activities applied, forming a backbone for tracking repair processes.

Operator and Aircraft

One operator oversees several aircraft, forming a 1-to-many structure. For example, carriers like Delta or Lufthansa handle numerous planes under their control. Each plane carries a distinct tail number. This identifier plays a key role in monitoring safety during flights. The system uses that number as the main reference point.

Aircraft Model

Keeping data unique requires including details like model, manufacturer and engine type.

MaintenanceTask and Part

Here lie the main tables. These hold idealized figures the time tasks ought to require, measure din hours, while also showing part prices.

Associative Entities (Junction Tables)

To resolve many-to-many relationships and capture transaction-specific data, two critical associative entities are utilized:

WorkOrderTask

A single work order may include several tasks. Actual hours appear here, separate from those projected in the main task record, enabling review of workforce performance. Though different in origin, both time measures support evaluation over time.

WorkOrderPart

During a given maintenance activity, tracking occurs for components used. Quantity used is tracked alongside cost per unit at that exact time.This value remains fixed regardless of later market shifts. Price changes afterward have no effect on earlier entries.

	Primary Key (PK)	**Foreign Key (FK)**	**Relationship Logic**
Operator	operator_id	None	Customer (Aircraft owner/ Airline)
AircraftModel	model_id	None	e.g. A320-200
Aircraft	aircraft_id	operator_id, model_id	Specific Aircraft with tail number
WorkOrder	workorder_id	aircraft_id	Records a specific maintenance event for one aircraft
MaintenanceTask	task_id	None	The master list of all certified repair procedures
WorkOrderTask	(workorder_id, task_id)	workorder_id, task_id	Tracks specific labor performed during a work order
Part	part_id	None	The master inventory catalog of available components
WorkOrderPart	(workorder_id, part_id)	workorder_id, part_id	Records parts consumed and their historical cost

Integrity and Normalization Details

Referential Integrity

Every foreign key is governed by strict constraints. For example, an `Aircraft` cannot be deleted if it has active `WorkOrders` attached, ensuring a permanent safety trail.

Composite Keys

The associative entities (`WorkOrderTask` and `WorkOrderPart`) utilize composite primary keys (the combination of the two IDs they link). This ensures that the same part or task cannot be erroneously duplicated within the same work order record.

Data Consistency

By separating the `Operator` and `Model` into their own tables, any change (such as an airline rebranding) only needs to be updated in one row, which then propagates through the entire `Aircraft` fleet via the foreign key relationship.

3. Database Implementation

3.1 DDL SQL Statements

Putting the EER model into practice, I shaped it into a working database structure. While starting out by building tables through MySQL Workbench's visual tools, I later focused on crafting a precise DDL script - making sure the setup could be rebuilt step by step using code. I created eight central tables, each given suitable data formats and created the respective relationships using primary and foreign keys.

```
-- 1. PHYSICAL SCHEMA DESIGN (DDL)
--
##############################################################################
#
USE mydb;
SET FOREIGN_KEY_CHECKS = 0;

-- I defined the core infrastructure tables
CREATE TABLE Operator (
    operator_id INT AUTO_INCREMENT PRIMARY KEY,
    operator_name VARCHAR(100) NOT NULL,
    country VARCHAR(50)
);

CREATE TABLE AircraftModel (
    model_id INT AUTO_INCREMENT PRIMARY KEY,
    manufacturer VARCHAR(50),
    model_name VARCHAR(50),
    engine_type VARCHAR(50)
);
```

```
CREATE TABLE MaintenanceTask (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    task_name VARCHAR(100),
    task_category VARCHAR(50),
    estimated_hours DECIMAL(10,2)
);

CREATE TABLE Part (
    part_id INT AUTO_INCREMENT PRIMARY KEY,
    part_number VARCHAR(50) UNIQUE,
    part_name VARCHAR(100),
    manufacturer VARCHAR(50),
    unit_cost DECIMAL(10,2)
);

-- I implemented the asset and transaction tracking tables
CREATE TABLE Aircraft (
    aircraft_id INT AUTO_INCREMENT PRIMARY KEY,
    tail_number VARCHAR(20) UNIQUE,
    operator_id INT,
    model_id INT,
    year_of_manufacture INT,
    status VARCHAR(50) DEFAULT 'Active',
    FOREIGN KEY (operator_id) REFERENCES Operator(operator_id),
    FOREIGN KEY (model_id) REFERENCES AircraftModel(model_id)
);

CREATE TABLE WorkOrder (
    workorder_id INT AUTO_INCREMENT PRIMARY KEY,
    aircraft_id INT,
    opened_date DATE,
    status ENUM('Open', 'Closed', 'In Progress'),
    FOREIGN KEY (aircraft_id) REFERENCES Aircraft(aircraft_id)
);

-- I created junction tables to manage many-to-many relationships
CREATE TABLE WorkOrderTask (
    workorder_id INT,
    task_id INT,
    actual_hours DECIMAL(10,2),
    PRIMARY KEY (workorder_id, task_id),
    FOREIGN KEY (workorder_id) REFERENCES WorkOrder(workorder_id),
    FOREIGN KEY (task_id) REFERENCES MaintenanceTask(task_id)
);

CREATE TABLE WorkOrderPart (
    workorder_id INT,
    part_id INT,
    quantity_used INT,
    unit_cost_at_time DECIMAL(10,2),
    PRIMARY KEY (workorder_id, part_id),
    FOREIGN KEY (workorder_id) REFERENCES WorkOrder(workorder_id),
    FOREIGN KEY (part_id) REFERENCES Part(part_id)
);

SET FOREIGN_KEY_CHECKS = 1;
```

3.2 Dummy Data Generation and Advanced Objects

I populated the database with a high-fidelity dataset including 15 global operators, 500 aircraft, and over 600 maintenance events. Furthermore, I developed advanced database objects—including specialized Indexes for performance, Views for financial reporting, Triggers for automated status updates, and Stored Procedures for standardized maintenance entry. These objects ensure the database functions as an active enterprise system rather than a static repository.

```
--
##############################################################################
#
-- 2. DATA POPULATION (DML)
--
##############################################################################
#

-- I seeded 15 Operators and 12 Aircraft Models
INSERT INTO Operator (operator_name, country) VALUES
('Lufthansa Airlines', 'Germany'), ('Lufthansa CityLine', 'Germany'),
('Lufthansa Cargo', 'Germany'),
('SWISS International', 'Switzerland'), ('Austrian Airlines', 'Austria'),
('Eurowings', 'Germany'),
('Brussels Airlines', 'Belgium'), ('Delta Air Lines', 'USA'), ('United
Airlines', 'USA'),
('Emirates', 'UAE'), ('Qatar Airways', 'Qatar'), ('Air France', 'France'),
('Singapore Airlines', 'Singapore'), ('NetJets', 'USA'), ('Wheels Up',
'USA');

INSERT INTO AircraftModel (manufacturer, model_name, engine_type) VALUES
('Boeing', '747-8', 'Turbofan'), ('Airbus', 'A380-800', 'Turbofan'),
('Boeing', '777F', 'Turbofan'),
('Airbus', 'A321F', 'Turbofan'), ('Airbus', 'A320neo', 'Turbofan'),
('Airbus', 'A350-900', 'Turbofan'),
('Boeing', '787-9', 'Turbofan'), ('Embraer', 'E190', 'Turbofan'),
('Bombardier', 'CRJ-900', 'Turbofan'),
('Gulfstream', 'G650', 'Jet'), ('Cessna', 'Citation Latitude', 'Jet'),
('Pilatus', 'PC-12', 'Turboprop');

-- I used CROSS JOINs to generate a massive dataset of 400 parts and 500
aircraft
INSERT INTO Part (part_number, part_name, manufacturer, unit_cost)
SELECT CONCAT('PN-', 10000 + (a.n + b.n*10 + c.n*100)),
ELT(FLOOR(1 + RAND()*6), 'Blade', 'Pump', 'Seal', 'Tire', 'Filter',
'Sensor'),
ELT(FLOOR(1 + RAND()*3), 'Honeywell', 'GE Aviation', 'Safran'), (50 +
RAND()*15000)
FROM (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3) a
CROSS JOIN (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION
SELECT 9) b
CROSS JOIN (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION
SELECT 9) c;

INSERT INTO Aircraft (tail_number, operator_id, model_id,
year_of_manufacture)
```

```
SELECT CONCAT('D-A', ELT(FLOOR(1+RAND()*4), 'X', 'Y', 'L', 'H'), a.n, b.n,
c.n),
(SELECT operator_id FROM Operator ORDER BY RAND() LIMIT 1),
(SELECT model_id FROM AircraftModel ORDER BY RAND() LIMIT 1),
FLOOR(2010 + RAND()*14)
FROM (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT
4) a
CROSS JOIN (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION
SELECT 9) b
CROSS JOIN (SELECT 0 n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION
SELECT 9) c
LIMIT 500;

--
##########################################################################
#
-- 3. ADVANCED OBJECTS (INDEXES, VIEWS, TRIGGERS, PROCEDURES)
--
##########################################################################
#
```

indexes

```
-- I created indexes to optimize search performance for the dashboard
CREATE INDEX idx_aircraft_tail ON Aircraft(tail_number);
CREATE INDEX idx_workorder_status ON WorkOrder(status);
```

views

```
-- I developed views to aggregate maintenance costs per aircraft
CREATE OR REPLACE VIEW view_aircraft_maintenance_costs AS
SELECT a.tail_number, o.operator_name, COUNT(DISTINCT wo.workorder_id) AS
total_orders,
ROUND(SUM(wop.quantity_used * wop.unit_cost_at_time), 2) AS
total_parts_cost
FROM Aircraft a JOIN Operator o ON a.operator_id = o.operator_id
LEFT JOIN WorkOrder wo ON a.aircraft_id = wo.aircraft_id
LEFT JOIN WorkOrderPart wop ON wo.workorder_id = wop.workorder_id
GROUP BY a.aircraft_id;
```

triggers

```
-- I implemented a trigger to automatically update aircraft status upon
maintenance entry
DELIMITER //
CREATE TRIGGER trg_after_workorder_insert
AFTER INSERT ON WorkOrder
FOR EACH ROW
BEGIN
    UPDATE Aircraft SET status = 'In Maintenance' WHERE aircraft_id =
NEW.aircraft_id;
END //

-- I created a trigger to ensure cost consistency by pulling prices from
the Part catalog
CREATE TRIGGER trg_auto_fill_part_cost
BEFORE INSERT ON WorkOrderPart
FOR EACH ROW
BEGIN
    SET NEW.unit_cost_at_time = (SELECT unit_cost FROM Part WHERE part_id =
NEW.part_id);
```

```
END //
DELIMITER ;
```

stored procedure

```
-- I developed a stored procedure to standardize the opening of new work
orders
DELIMITER //
CREATE PROCEDURE sp_open_standard_maintenance(IN p_aircraft_id INT)
BEGIN
    INSERT INTO WorkOrder (aircraft_id, opened_date, status)
    VALUES (p_aircraft_id, NOW(), 'Open');
    SET @new_id = LAST_INSERT_ID();
    INSERT IGNORE INTO WorkOrderTask (workorder_id, task_id, actual_hours)
    VALUES (@new_id, 1, 2.0);
    SELECT CONCAT('WorkOrder #', @new_id, ' opened for Aircraft ID ',
p_aircraft_id) AS Result;
END //
DELIMITER ;
```

4. Database Deployment Documentation

4.1 AWS RDS Configuration Details

Database Engine: MySQL 8.0.x

Instance Class: db.t3.micro (1 vCPU, 1GB RAM)

Storage: 20 GB General Purpose SSD (gp2)

Region: us-west-1 (N. California)

Endpoint: database-mis686tp.cb6ie0ayqmug.us-west-1.rds.amazonaws.com

Port: 3306

Database Configuration:

Database Name: mydb

Username: admin

Password: Ichschaffdas1--

Character Set: UTF8MB4

Storage Engine: InnoDB

4.2 Implementation Procedures

Before building the database I set up a custom Parameter Group. It helped bypass a restriction in the standard configuration. Since triggers and stored routines cannot be enabled under default options, adjusting these settings became necessary. With that change in place, further setup could proceed.

Starting at the Parameter Groups section, I clicked on create to begin the process. Choosing the mysql 8.0 engine type came next. With the fresh group open, a search targeted log_bin_trust_function_creators. Adjusting its setting to 1 happened before saving took place. Then I executed the SQL code.

AWS RDS Instance Creation

I navigated to the RDS dashboard and clicked "Create Database". I selected MySQL as the engine and chose the Free Tier template to stay within the resource limits. I set the DB Instance Identifier to database-mis686tp and configured the master username and password. Under Connectivity, I enabled Public Access so I could connect from the university or my home.

In the Additional Configuration, I manually selected the custom Parameter Group I created in Step 1.

Security Group & Firewall Configuration After the database was initialized, I configured the Security Group to act as a firewall. I accessed the inbound rules for the security group I created.I modified the existing rule to allow MySQL/Aurora (Port 3306) traffic. I changed the source to 0.0.0.0/0. While I understand this isn't standard for production, I did this to ensure I could connect from any IP address as I moved between home and campus.

Once the instance status changed to available, I connected via MySQL Workbench using my endpoint and credentials. I then ran my full script to build the architecture.

5. Analytical Questions and Dashboard Creation

The Aviation Maintenance Enterprise Dashboard provides a real-time overview of fleet health, financial capital, and logistics performance. To generate these insights, I used an interactive Python environment (Jupyter/Google Colab).

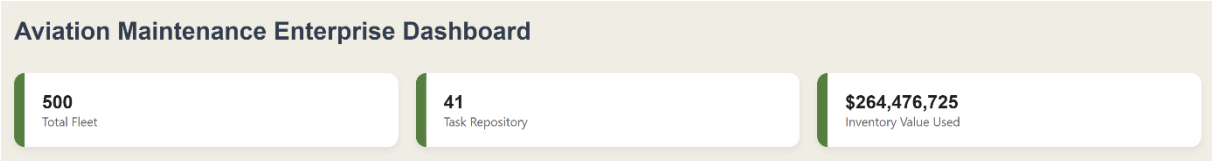
Technical Implementation

To connect Python with the SQL database, I used the mysql-connector-python library alongside pandas. I linked it with the AWS RDS instance through specific code execution. Data flowed into dataframes once the query ran without errors. Visuals formed afterwards, built using Plotly Express. The final output was exported as a standalone HTML reporting file.

Dashboard Generation

```
import mysql.connector

import pandas as pd
import plotly.express as px

# 1. Database Connection
conn = mysql.connector.connect(
    host='database-mis686tp.cb6ie0ayqmug.us-west-1.rds.amazonaws.com',
    user='admin', password='Ichschaffdas1--', database='mydb'
)

# 2. Execute Query and Generate Chart
query = "SELECT task_category, COUNT(task_id) as task_count FROM
MaintenanceTask GROUP BY 1"
df = pd.read_sql(query, conn)
fig1 = px.bar(df, x='task_count', y='task_category', orientation='h',
title="Task Volume")

# 3. Save Dashboard to HTML
dashboard_html = fig1.to_html(full_html=True)
with open("report.html", "w") as f:
    f.write(dashboard_html)
```

Analytical Questions and Dashboard



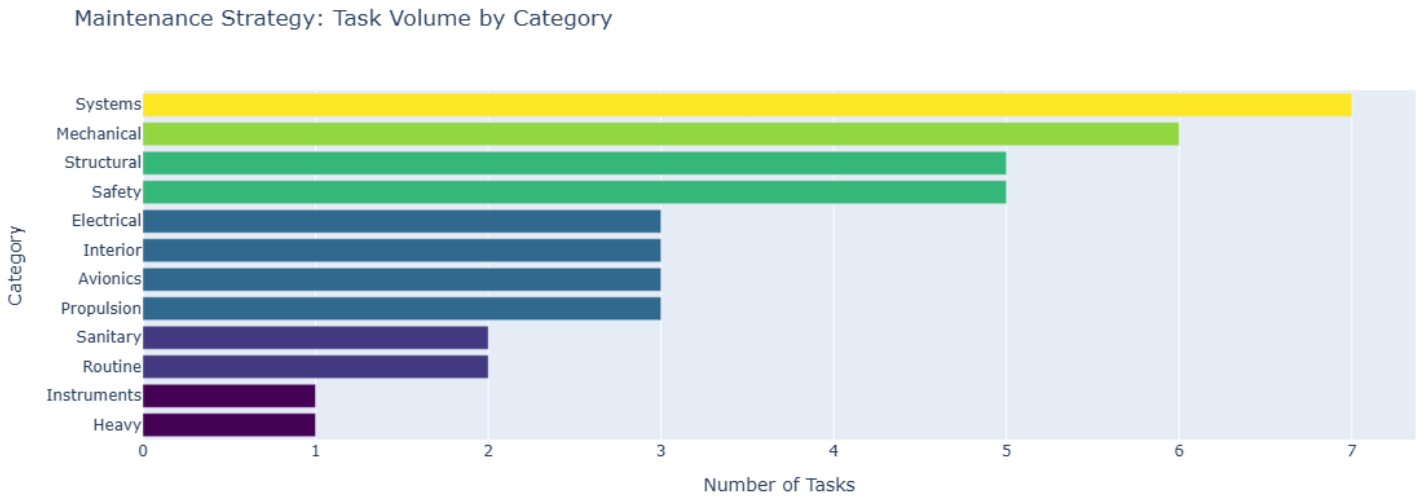
Q1: Maintenance Strategy – Task Volume by Category

SQL Query

```
SELECT task_category, COUNT(task_id) as task_count
FROM MaintenanceTask
GROUP BY task_category;
```

Business Insight

This type of horizontal bar graph illustrates how maintenance labor is classified. It becomes clear which tasks (such as system repairs vs. propulsion-related tasks) are most dominant in determining the operational need. It should be noted in the context of the graph provided that the largest portion of the tasks performed are related to system maintenance.



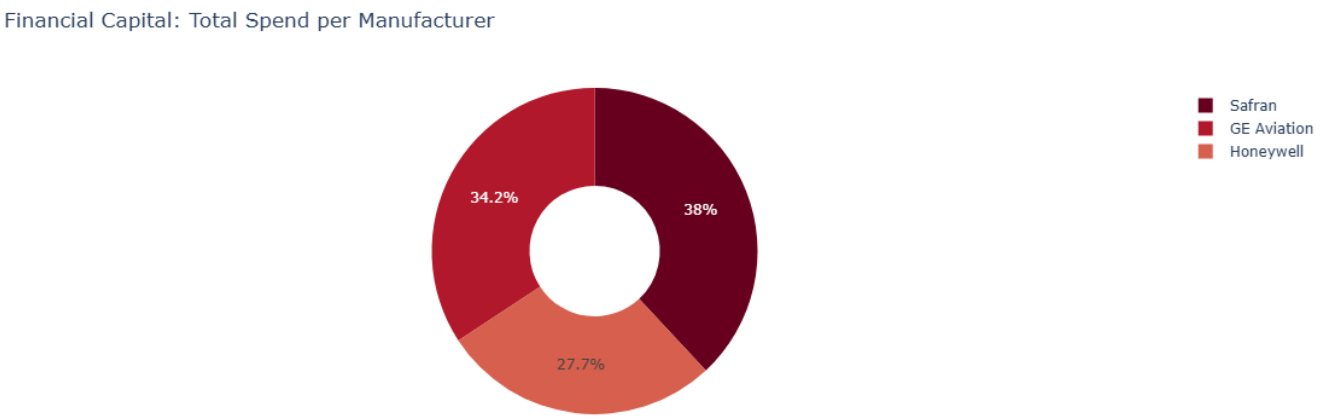
Q2: Financial Capital – Total Spend per Manufacturer

SQL Query

```
SELECT p.manufacturer, SUM(wop.quantity_used * wop.unit_cost_at_time) as
Total_Spend
FROM Part p
JOIN WorkOrderPart wop ON p.part_id = wop.part_id
GROUP BY p.manufacturer;
```

Business Insight

This ring chart shows the breakdown of total financial investment into part-manufacturing companies. This emphasizes dependencies in procurement – for example, the significant investment of *38% with Safran* may lead decision makers to negotiate heavily with this supplier regarding the cost of maintenance.



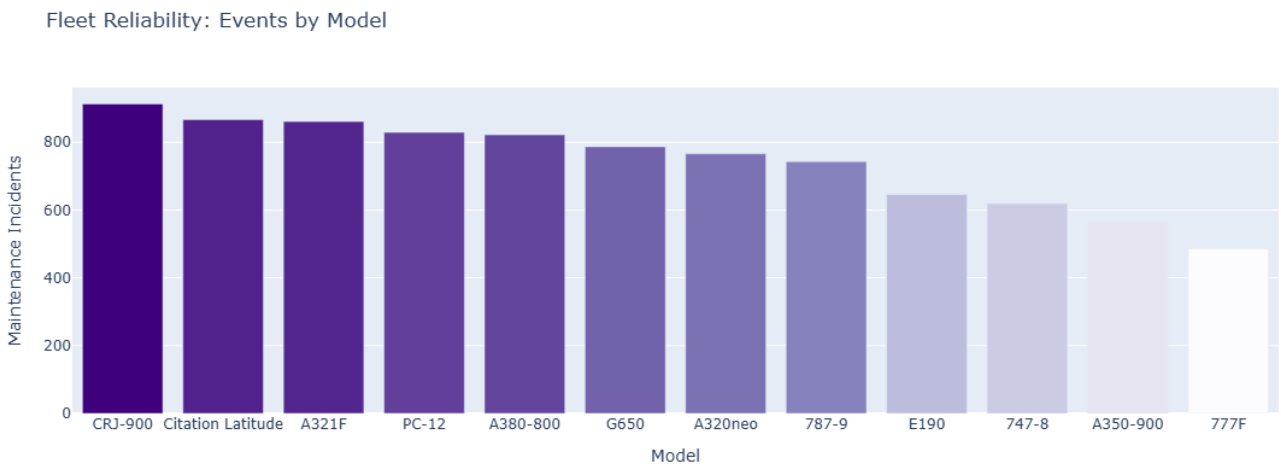
Q3: Fleet Reliability – Events by Aircraft Model

SQL Query:

```
SELECT am.model_name, COUNT(wo.workorder_id) as Events
FROM AircraftModel am
JOIN Aircraft a ON am.model_id = a.model_id
JOIN WorkOrder wo ON a.aircraft_id = wo.aircraft_id
GROUP BY am.model_name;
```

Business Insight

By classifying the maintenance incidents based on aircraft model, one may identify what types of aircraft tend to be more prone to technical issues. If the Airbus A320neo happens to accrue significantly more events compared to the Boeing 787-9, this could indicate its more intense flight schedule or early reliability issues that may call for a review of the preventive maintenance plan for this particular model. It's observable that the model CRJ-900 has the highest level of maintenance incidents and the Boeing 777F has the least.



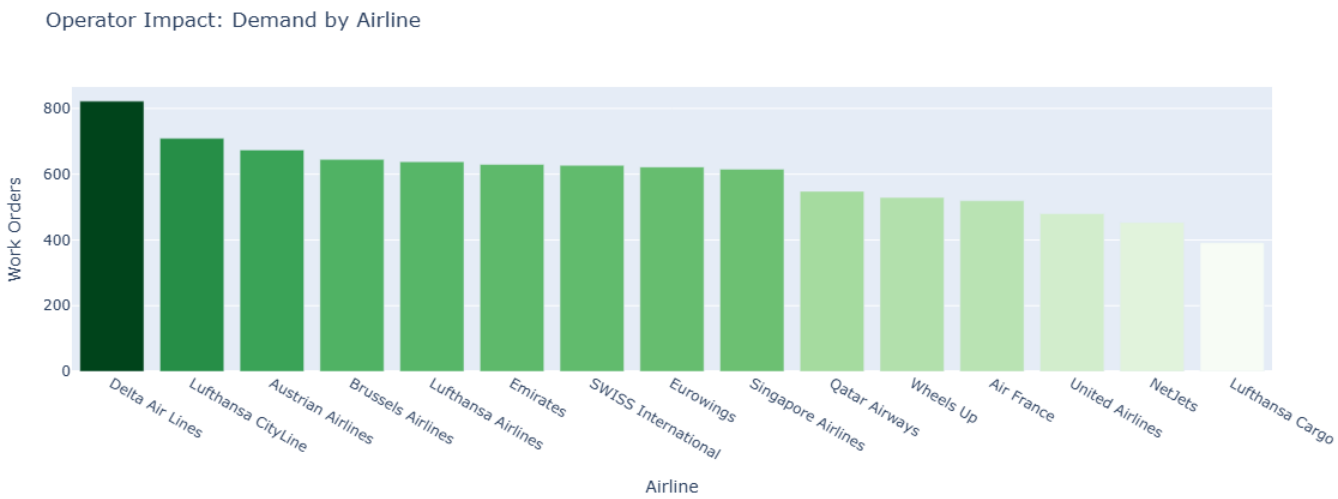
Q4: Operator Impact – Demand by Airline

SQL Query

```
SELECT o.operator_name, COUNT(wo.workorder_id) as Activity
FROM Operator o
JOIN Aircraft a ON o.operator_id = a.operator_id
JOIN WorkOrder wo ON a.aircraft_id = wo.aircraft_id
GROUP BY o.operator_name;
```

Business Insight

This graph reveals airline operators who are responsible for the greatest demands on maintenance. In the MRO environment, this helps understand how to prioritize relationships with our clients. Here, the airline demanding the most is Delta Airlines.

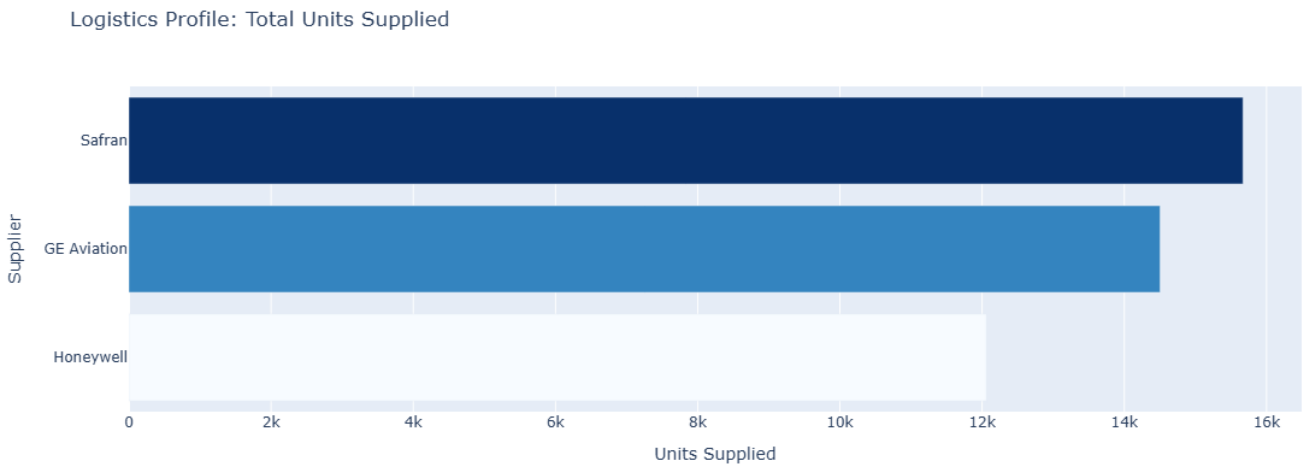


Q5: Logistics Profile – Total Units Supplied

SQL Query

```
SELECT p.manufacturer, SUM(wop.quantity_used) as Part_Qty
FROM Part p
JOIN WorkOrderPart wop ON p.part_id = wop.part_id
GROUP BY p.manufacturer;
```

Business Insight: Compared to the financial chart, this logistics profile focuses intently on actual volume. This is the number of parts that pass through the supply chain from each manufacturer. This is important in a warehouse setting because the manufacturers with the most parts, regardless of cost, require greater storage space and greater service frequency.



Q6: Inventory Focus – High-Value Spare Parts Analysis

SQL Query

```
SELECT p.part_name, p.manufacturer, SUM(wop.quantity_used) as Qty,
       SUM(wop.quantity_used * wop.unit_cost_at_time) as Total_Cost
FROM Part p
JOIN WorkOrderPart wop ON p.part_id = wop.part_id
GROUP BY 1, 2
ORDER BY Total_Cost DESC
LIMIT 10;
```

Business Insight

This data table highlights the ten components that consume the most budget. By focusing on these essential components such as Turbine Blades or Engine Sensors, for instance, one may apply Just-In-Time inventory management. The aim would be to reduce tied-down capital in the form of expensive spares while ensuring that aircraft are always accessible and never grounded because of the lack of needed parts.

Financial Focus: Top 10 High-Value Spare Parts

Part Name	Manufacturer	Units Used (Qty)	Total Inventory Value (USD)
Filter	GE Aviation	2,985.00	$21,436,123.04
Blade	Safran	3,134.00	$21,339,093.85
Blade	Honeywell	2,741.00	$18,211,460.16
Blade	GE Aviation	2,621.00	$18,011,932.29
Filter	Safran	2,729.00	$17,818,676.91
Seal	Safran	2,418.00	$17,103,739.80
Pump	Safran	2,480.00	$16,626,120.83
Seal	GE Aviation	2,459.00	$15,884,569.80
Tire	GE Aviation	2,687.00	$15,493,609.72
Seal	Honeywell	2,268.00	$14,836,434.31

Technologies and Tools

Category	Technology	Purpose
Database	MySQL 8.0	Cloud relational database management
Cloud	AWS RDS	Managed hosting for database instance
Language	Python 3	Backend automation and data analytics
Data Analysis	Pandas	SQL-to-DataFrame manipulation
Visualization	Plotly Express	Interactive chart and dashboard generation
Driver	MySQL Connector	Secure bridge between Python and AWS
Environment	Jupyter / Colab	Interactive development and code execution

6. Conclusion

This term project illustrates how to build an effective relational database. Through hosting the database on AWS RDS, I created an integrated system showcasing complex relationships between operators, aircraft models, maintenance work requests, and high-value component logistics. With the help of Python code, I created valuable dashboards crucial for MRO decision makers.

Project Repository

https://github.com/gollmichamarkus/aviation-maintenance-dashboard

Dashboard link

gollmichamarkus.github.io/aviation-maintenance-dashboard/report.html

Code generated with Google Gemini

MIS686 Enterprise Database Management - Fall 2025